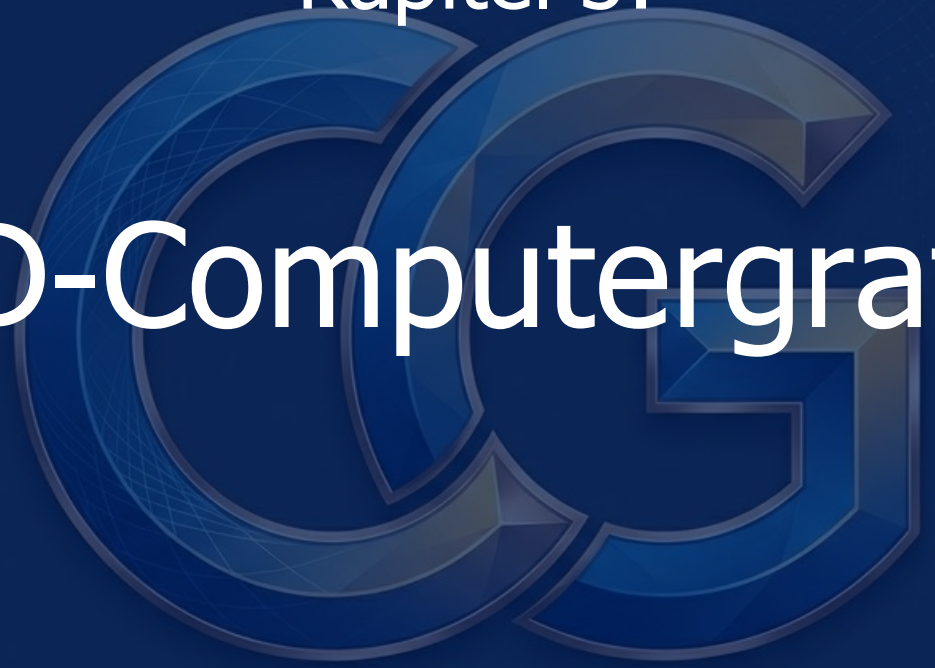
Vorlesung: Grundlagen der Computergrafik

Kapitel 3:



3D-Computergrafik

Vorlesung im Sommersemester 2026
Prof. Dr. Stephan Diehl, Informatik, Universität Trier



Organisatorisches

- Vorlesung nächste Woche „praktisch“
  - SVG + Transformationen
  - Heute Übung

Kapitelübersicht

- Kapitel 3.1: Die Rendering-Pipleline
- Kapitel 3.2: Transformationen
- Kapitel 3.3: Projektionen
- Kapitel 3.4: Beleuchtung
- Kapitel 3.5: Polygonnetze
- Kapitel 3.6: Texturen

Vorlesung: Grundlagen der Computergrafik

Kapitel 3: 3D-Computergrafik

Kapitel 3.1:

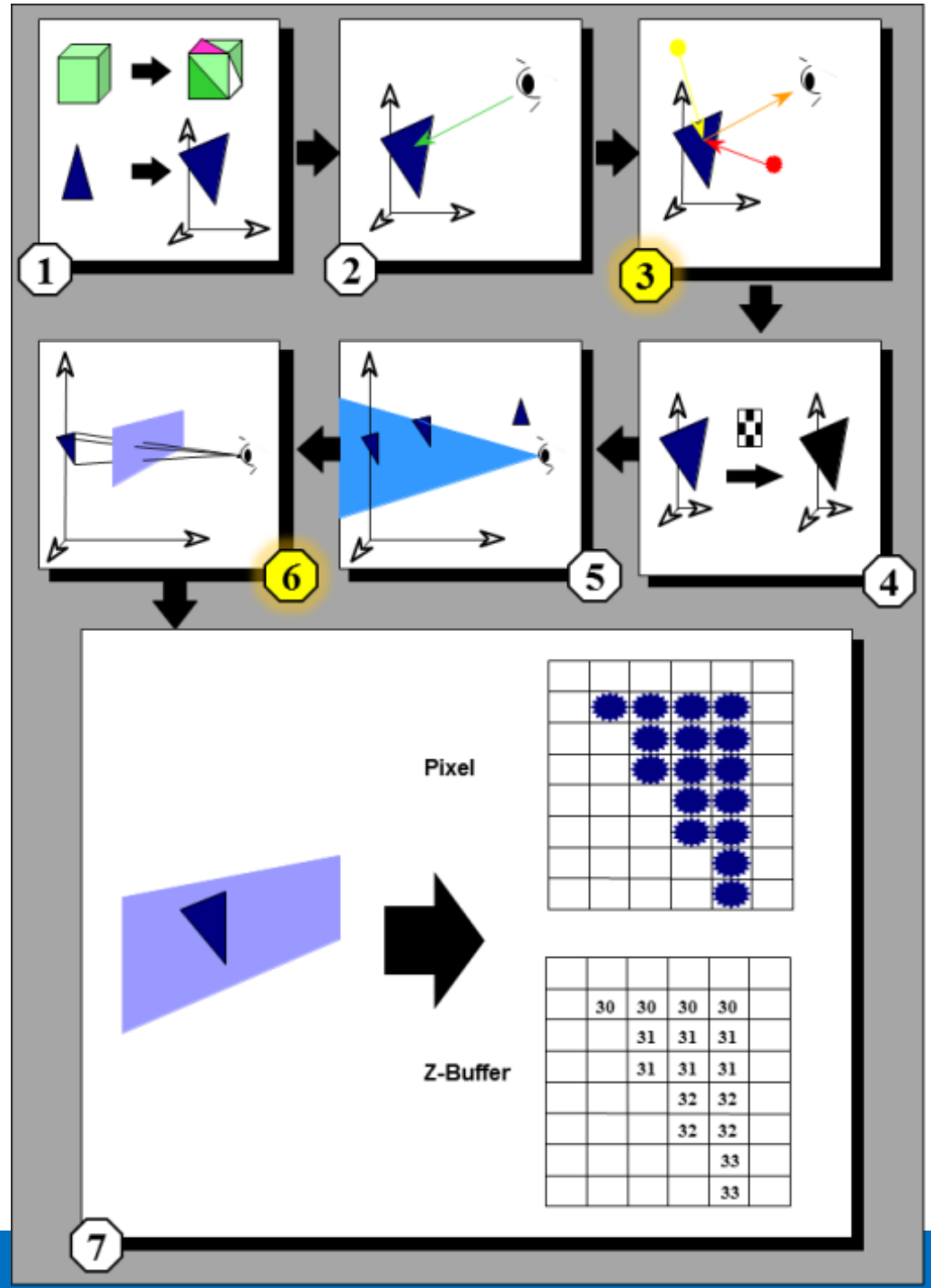
Die Rendering-Pipeline

Vorlesung im Sommersemester 2026
Prof. Dr. Stephan Diehl, Informatik, Universität Trier



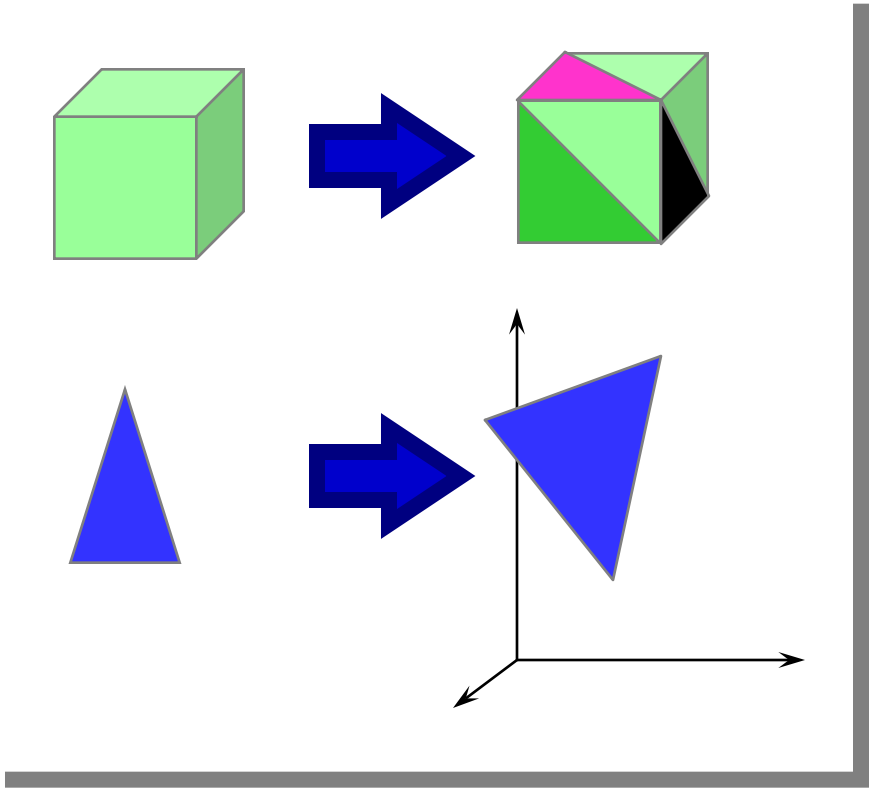
Rendering Pipeline

heute



Rendering Pipeline

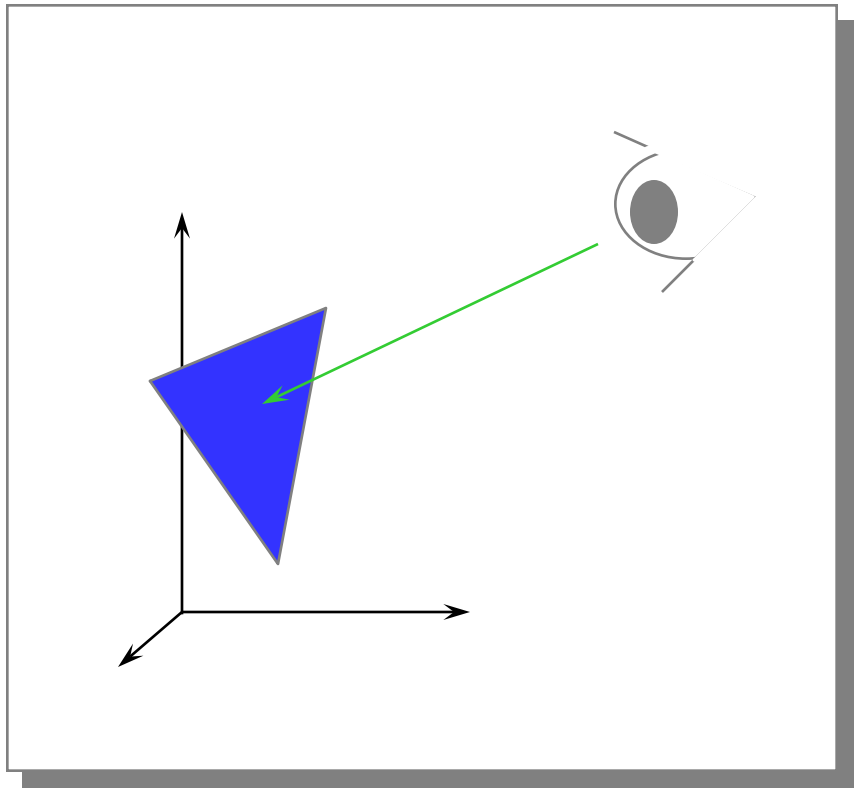
(1) Transform



Rendering Pipeline

(1) Transform

(2) Backface Cull

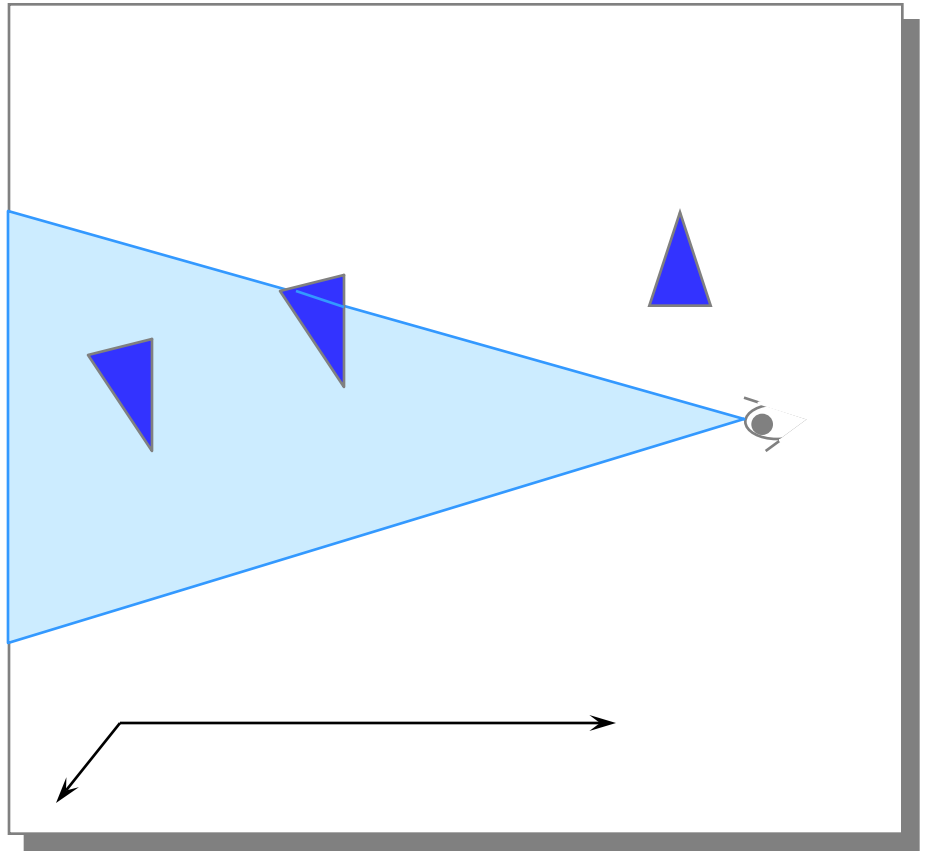


Rendering Pipeline

(1) Transform

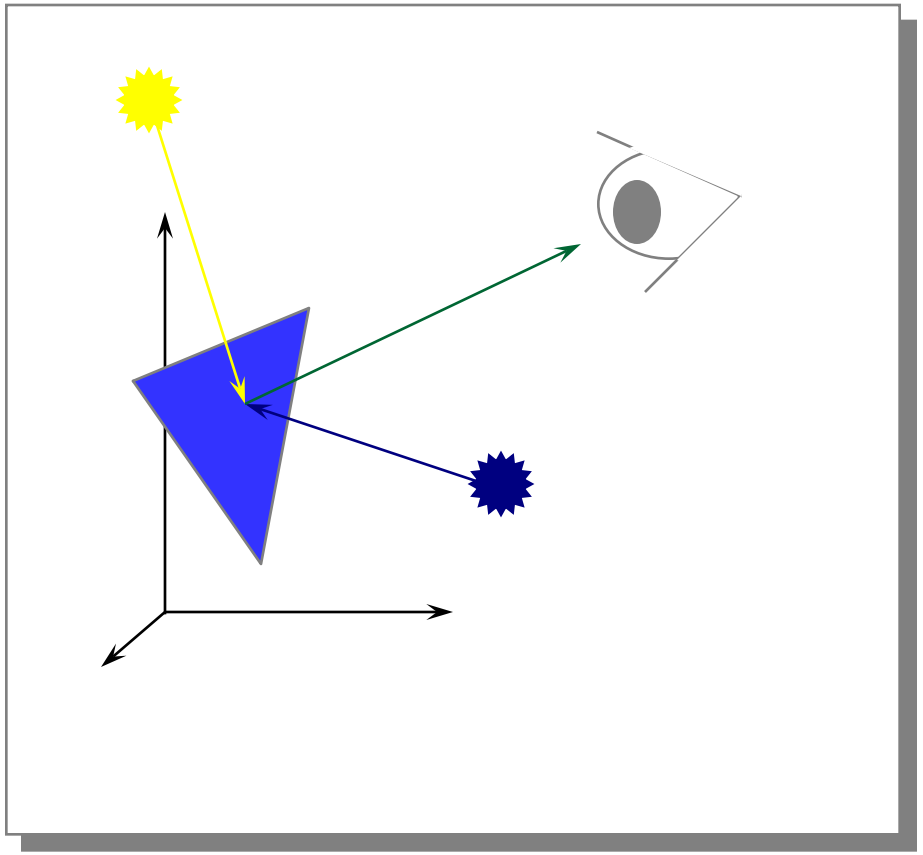
(2) Backface Cull

(3) View Cull and Clip



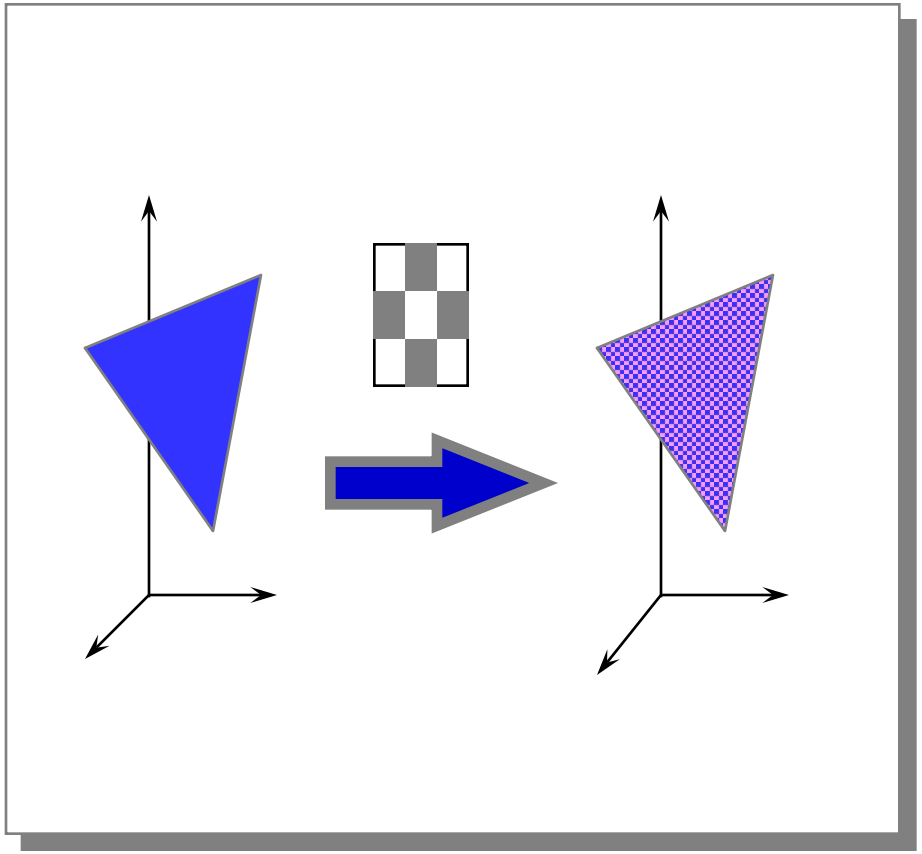
Rendering Pipeline

(1) Transform
(2) Backface Cull
(3) View Cull and Clip
(4) Lighting



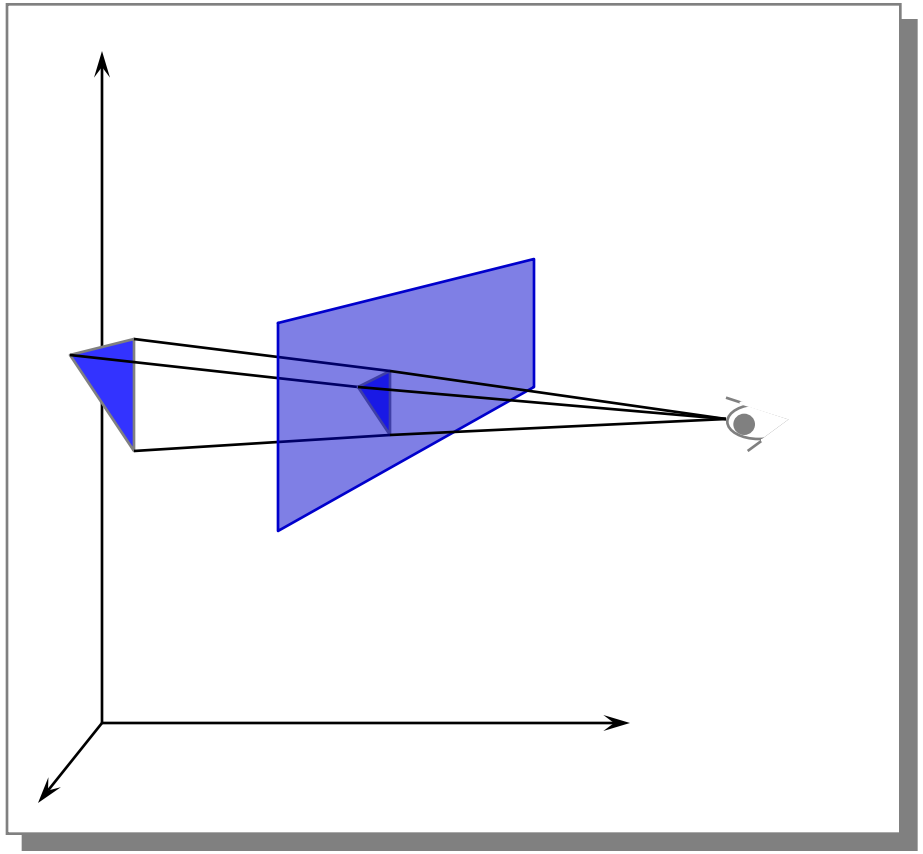
Rendering Pipeline

(1) Transform
(2) Backface Cull
(3) View Cull and Clip
(4) Lighting
(5) Texture Transform



Rendering Pipeline

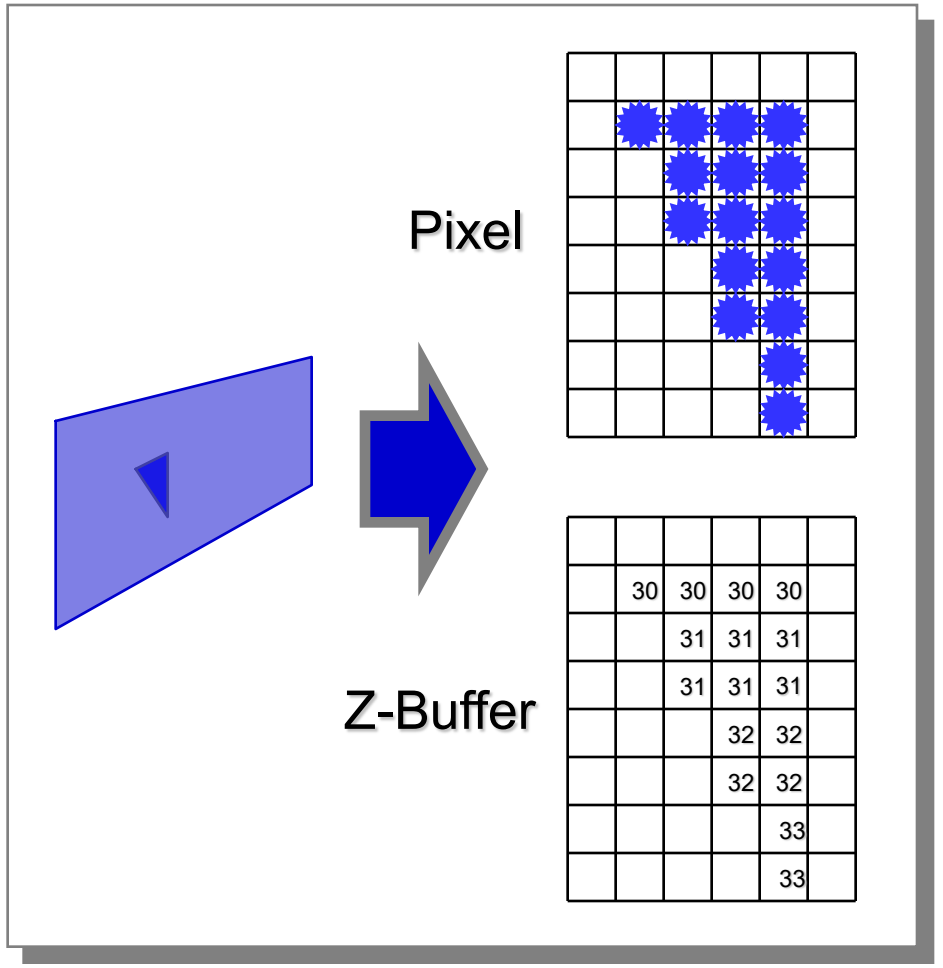
(1) Transform
(2) Backface Cull
(3) View Cull and Clip
(4) Lighting
(5) Texture Transform
(6) View Project



Rendering Pipeline

Information
pro Eckpunkt:

(1) Transform
(2) Backface Cull
(3) View Cull and Clip
(4) Lighting
(5) Texture Transform
(6) View Project
(7) Rasterize



Für alle Eckpunkte eines Dreiecks

vorgegeben:

- Position im lokalen Koordinatensystem
- Eigenfarbe

berechnet:

(1) Transform → • Position im globalen Koordinatensystem

(2) Backface Cull

(3) View Cull and Clip

(4) Lighting → • Farbe in Kombination mit Lichtquellen

(5) Texture Transform → • Position im Texturraum

(6) View Project → • Lage auf Projektionsebene

(7) Rasterize → • Pixelkoordinaten

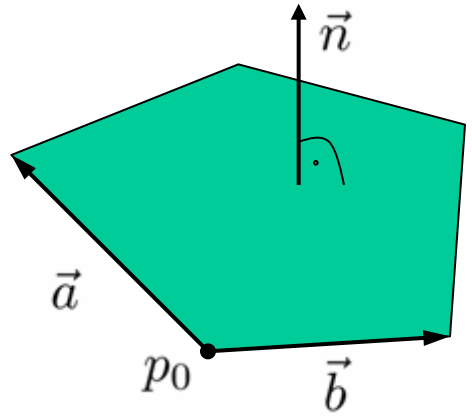
Polygon, Ebene, Flächennormale

auch:
dot-product
Punktprodukt
Skalarprodukt

Inneres Produkt: $\vec{a} \cdot \vec{b} = \sum_{i=1}^{n} a_i b_i$

Norm: $|\vec{a}| = \sqrt{\vec{a} \cdot \vec{a}}$

Winkel zwischen Vektoren: $\vec{a} \cdot \vec{b} = |\vec{a}||\vec{b}| cos\ \alpha$



$\vec{b}, \vec{a}$ und $\vec{n}$ bilden ein rechthändiges System

auch:
Kreuzprodukt

Vektorprodukt (hier nur 3D): $\vec{a} \times \vec{b} = \begin{pmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{pmatrix}$

$\vec{a} \times \vec{b}$ steht senkrecht auf der von $\vec{a}$ und $\vec{b}$ aufgespannten Ebene.

Seien $\vec{a}$ und $\vec{b}$ benachbarte gerichtete Kanten (siehe Abbildung):
Normale: $\vec{n} = \vec{b} \times \vec{a}$ (im Gegenuhrzeigersinn, zeigt nach "Außen")

Ebenengleichung: Alle Punkte $\vec{P}$ der Ebene erfüllen die Gleichung $\vec{n} \cdot \vec{p} = d$

Ist ein Punkt p_0, z.B. ein Eckpunkt des Polygons, gegeben, so kann man d direkt berechnen.

Vorlesung: Grundlagen der Computergrafik

Kapitel 3: 3D-Computergrafik



Kapitel 3.2:

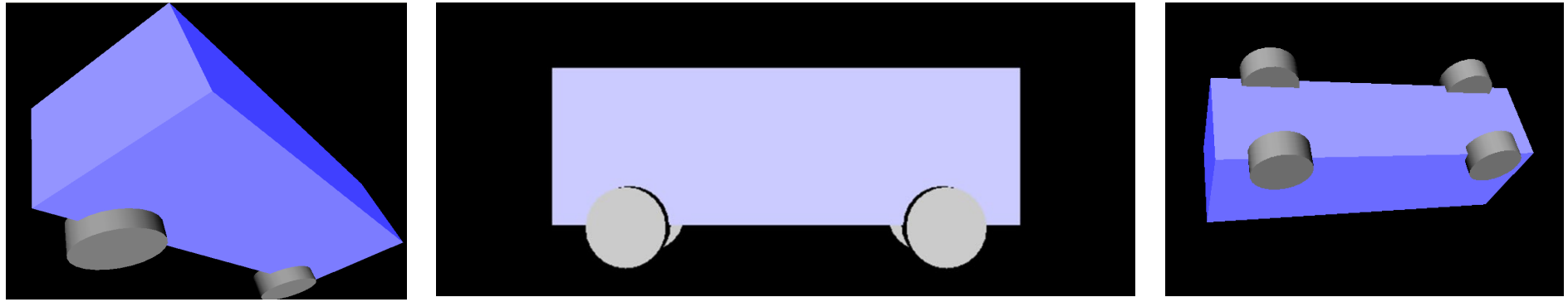
Transformationen

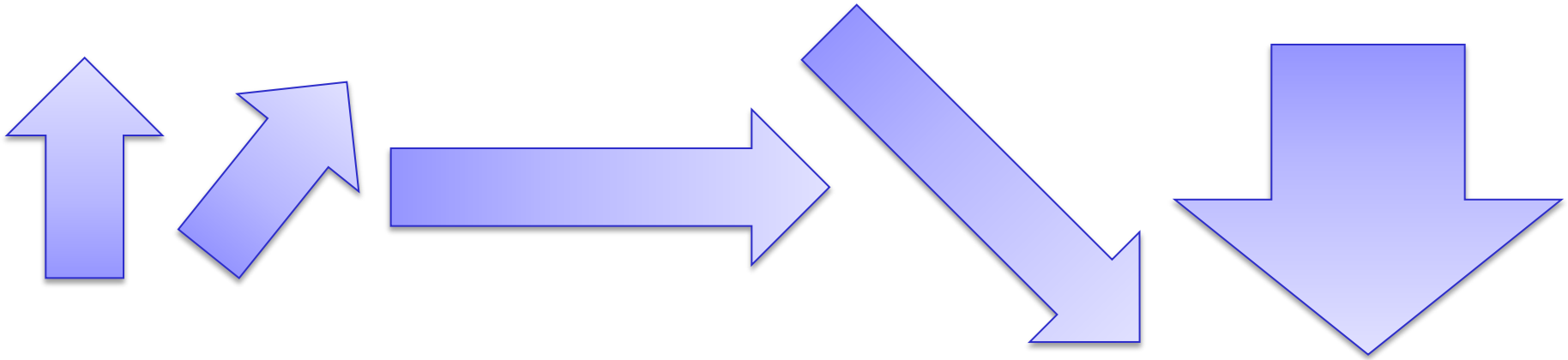
Vorlesung im Sommersemester 2026

Prof. Dr. Stephan Diehl, Informatik, Universität Trier

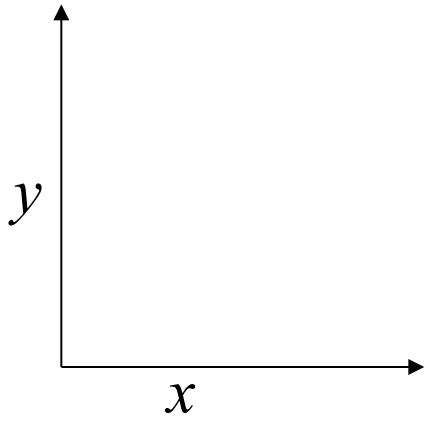


Transformationen in 2D und 3D

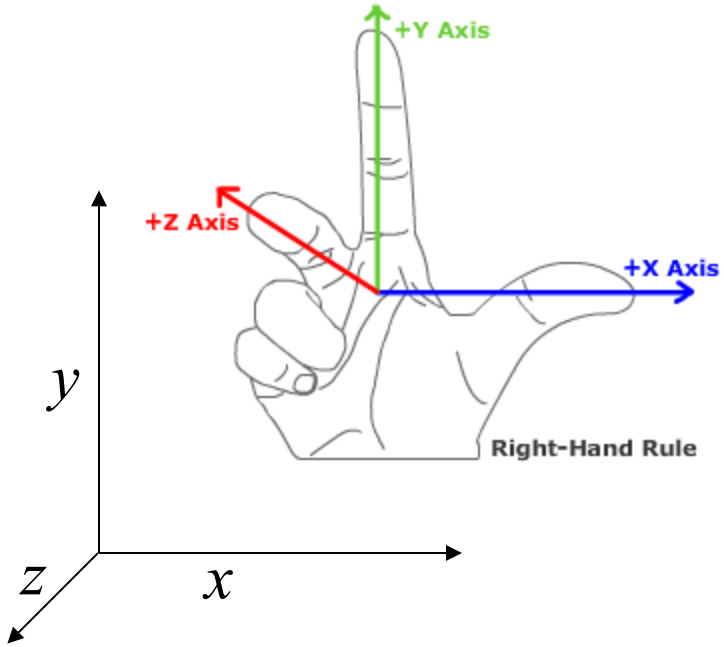




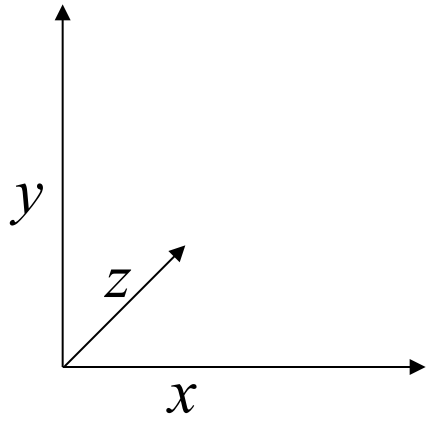
Koordinatensysteme



2D Koordinatensystem



Rechtshändiges 3D Koordinatensystem



Linkshändiges 3D Koordinatensystem

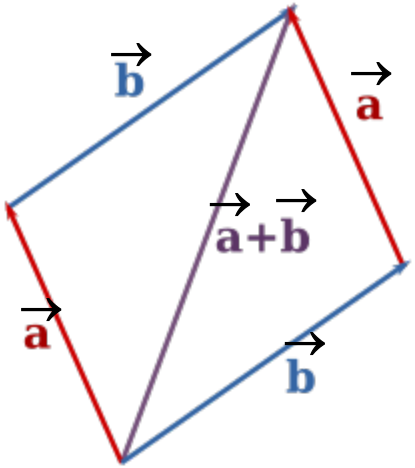
x → Daumen, y → Zeigefinger, z → Mittelfinger

Vektoren

$$\mathrm{a} = \begin{pmatrix} a_1 \\ a_2 \end{pmatrix}$$

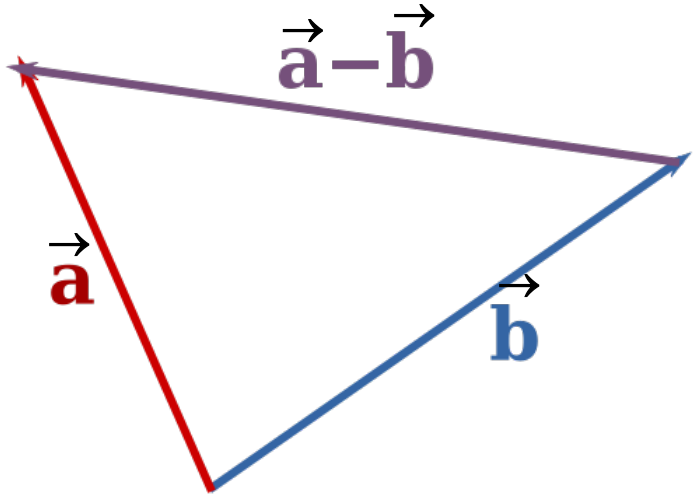
$$\mathrm{b} = \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}$$

- Vektoraddition



$$\vec{a} + \vec{b} = \begin{pmatrix} a_1 + b_1 \\ a_2 + b_2 \end{pmatrix}$$

- Vektorsubtraktion



$$\vec{a} - \vec{b} = \begin{pmatrix} a_1 - b_1 \\ a_2 - b_2 \end{pmatrix}$$

Matrizenmultiplikation



$$\begin{pmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \\ a_3 & b_3 & c_3 \end{pmatrix} * \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} a_1 * x + b_1 * y + c_1 * z \\ a_2 * x + b_2 * y + c_2 * z \\ a_3 * x + b_3 * y + c_3 * z \end{pmatrix}$$

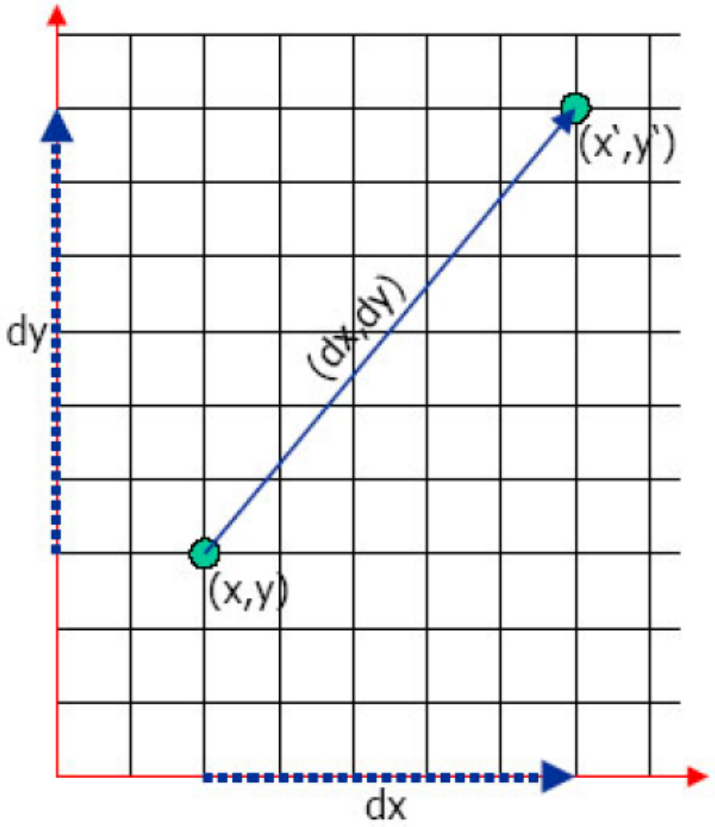
Translation (Verschiebung)

Verschiebung eines Punktes $P = (x, y, z)^T$ nach $P' = (x', y', z')^T$ um $T = (dx, dy, dz)^T$:

$$x' = x + dx$$
$$y' = y + dy$$
$$z' = z + dz$$

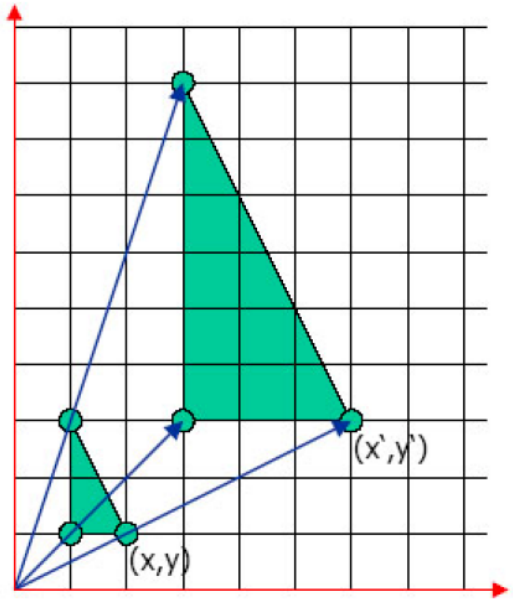
$$\begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = \begin{pmatrix} x \\ y \\ z \end{pmatrix} + \begin{pmatrix} dx \\ dy \\ dz \end{pmatrix}$$

$$P' = P + T$$



Skalierung (Vergrößern/Verkleinern)

Ein Objekt wird skaliert, indem jeder Punkt des Objektes skaliert wird.



$$x' = sx * x$$
$$y' = sy * y$$
$$z' = sz * z$$

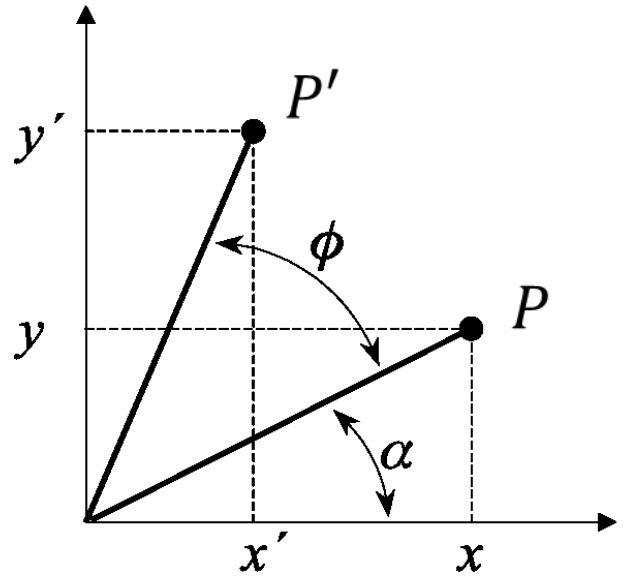
Häufig: $sx = sy = sz$

$$\begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = \begin{pmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & sz \end{pmatrix} * \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

$$P' = S * P$$

Rotation (Drehung)

2D: Drehung um den Ursprung um Winkel ϕ



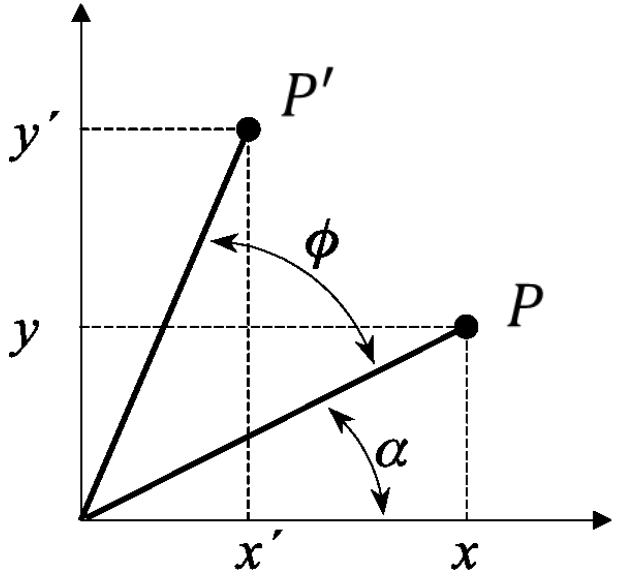
$$\sin(\alpha \pm \beta) = \sin\alpha\cos\beta \pm \sin\beta\cos\alpha$$
$$\cos(\alpha \pm \beta) = \cos\alpha\cos\beta \mp \sin\alpha\sin\beta$$

Rotation (Drehung)

2D: Drehung um den Ursprung um Winkel ϕ

$$
\begin{aligned}
x &= r * cos\ \alpha \\
y &= r * sin\ \alpha
\end{aligned}
$$

$$
\begin{aligned}
x' &= r * cos\ \alpha + \phi \\
&= r * cos\ \alpha * cos\ \phi - r * sin\ \alpha * sin\ \phi \\
&= x * cos\ \phi - y * sin\ \phi \\
y' &= r * sin\ \alpha + \phi \\
&= r * cos\ \alpha * sin\ \phi + r * sin\ \alpha * cos\ \phi \\
&= x * sin\ \phi + y * cos\ \phi
\end{aligned}
$$

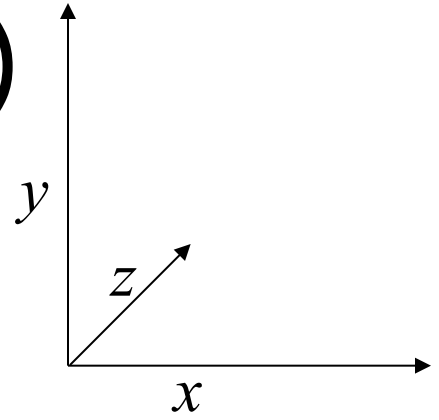


$$
\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} cos\ \phi & -sin\ \phi \\ sin\ \phi & cos\ \phi \end{pmatrix} * \begin{pmatrix} x \\ y \end{pmatrix}
$$

$$P' = R * P$$

$$
\begin{aligned}
\sin(\alpha \pm \beta) &= \sin\alpha\cos\beta \pm \sin\beta\cos\alpha \\
\cos(\alpha \pm \beta) &= \cos\alpha\cos\beta \mp \sin\alpha\sin\beta
\end{aligned}
$$

Rotation (Drehung)



3D: Drehung um eine der drei Koordinatenachsen

Drehung um Z-Achse (entspricht 2D-Drehung):

$$\begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = \begin{pmatrix} cos\ \phi & -sin\ \phi & 0 \\ sin\ \phi & cos\ \phi & 0 \\ 0 & 0 & 1 \end{pmatrix} * \begin{pmatrix} x \\ y \\ z \end{pmatrix} \qquad P' = R_z^\phi * P$$

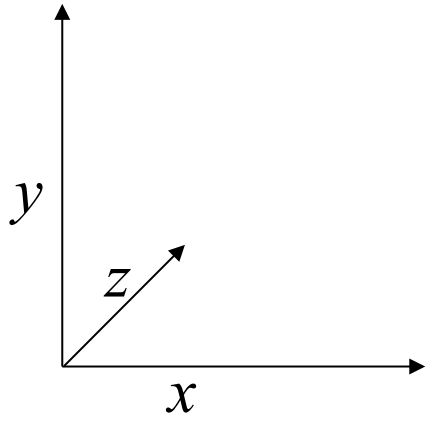
Drehung um Y-Achse:

$$\begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = \begin{pmatrix} cos\ \phi & 0 & sin\ \phi \\ 0 & 1 & 0 \\ -sin\ \phi & 0 & cos\ \phi \end{pmatrix} * \begin{pmatrix} x \\ y \\ z \end{pmatrix} \qquad P' = R_y^\phi * P$$

Drehung um X-Achse:

$$\begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & cos\ \phi & -sin\ \phi \\ 0 & sin\ \phi & cos\ \phi \end{pmatrix} * \begin{pmatrix} x \\ y \\ z \end{pmatrix} \qquad P' = R_x^\phi * P$$

Rotation (Drehung)



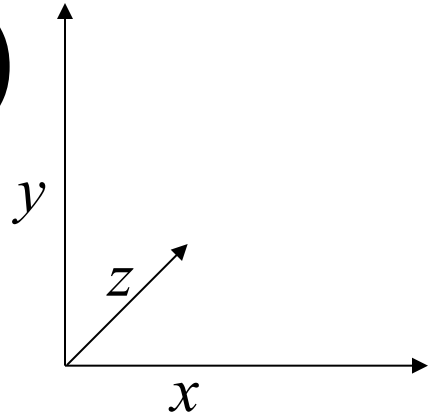
Eine beliebige Rotation setzt sich zusammen aus Rotationen um die drei Achsen:

$$P' = R_x^\alpha * R_y^\beta * R_z^\gamma * P$$

Da Matrizenmultiplikation assoziativ ist, können die drei Rotationsmatrizen ausmultipliziert und durch eine einzige ersetzt werden:

$$P' = R * P$$

Rotation (Drehung)



Spielt die Reihenfolge der Rotationen eine Rolle?

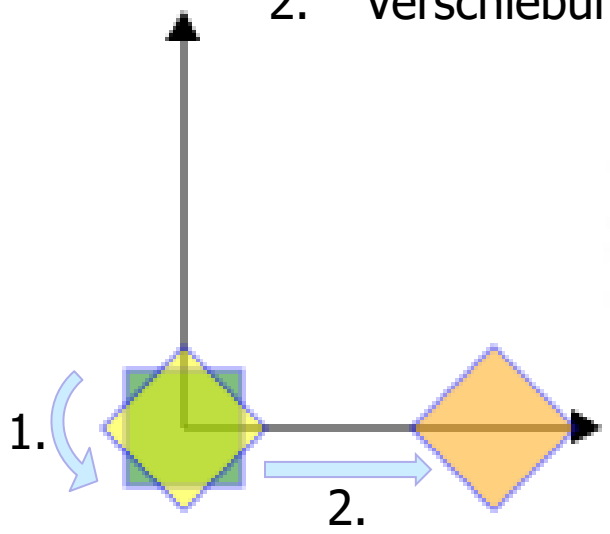
$$R_z^{\alpha} * R_y^{\beta} = \begin{pmatrix} cos\ \alpha * cos\ \beta & -sin\ \alpha & cos\ \alpha * sin\ \beta \\ sin\ \alpha * cos\ \beta & cos\ \beta & sin\ \alpha * sin\ \beta \\ -sin\ \beta & 0 & cos\ \beta \end{pmatrix} \xrightarrow{\alpha = \beta = 90°} \begin{pmatrix} 0 & -1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

$$R_y^{\beta} * R_z^{\alpha} = \begin{pmatrix} cos\ \alpha * cos\ \beta & -sin\ \alpha * cos\ \beta & sin\ \beta \\ sin\ \alpha & cos\ \alpha & 0 \\ -cos\ \alpha * sin\ \beta & -sin\ \alpha * sin\ \beta & cos\ \beta \end{pmatrix} \xrightarrow{\alpha = \beta = 90°} \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & -1 & 0 \end{pmatrix}$$

$\Rightarrow$ **Matrizenmultiplikation ist nicht kommutativ, d.h. die Reihenfolge, in der die Drehungen ausgeführt werden, spielt eine Rolle!**

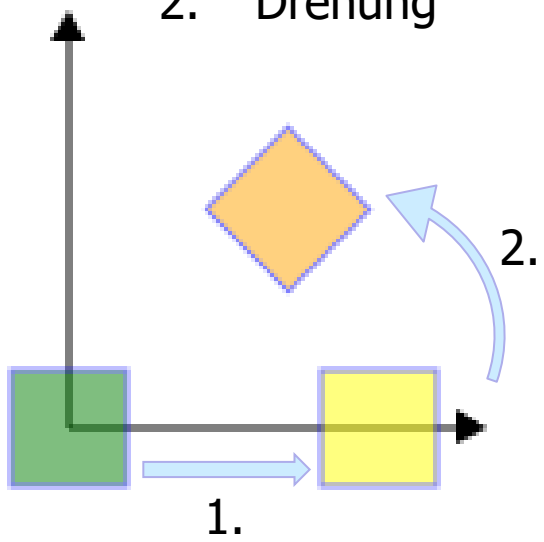
Transformationen sind meist nicht kommutativ

1. Drehung
2. Verschiebung



≠

1. Verschiebung
2. Drehung




```
<use  xlink:href="#quadrat" fill="green" />
<use  xlink:href="#quadrat" fill="yellow"
      transform="rotate(45)" />
<use  xlink:href="#quadrat" fill="orange"
      transform="translate(80),rotate(45)" />
```

```
<use  xlink:href="#quadrat" fill="green" />
<use  xlink:href="#quadrat" fill="yellow"
      transform="translate(80,0)" />
<use  xlink:href="#quadrat" fill="orange„
      transform="rotate(45),translate(80,0)" />
```

Komplexe Transformationen

Rotiere mit R, dann verschiebe mit T_1,
dann skaliere mit S und verschiebe mit T_2

$$P' = T_2 + S * (T_1 + R * P)$$

Wegen der Addition ist die Zusammenfassung der Matrizen nicht möglich.

Kann man Translation auch durch Matrizenmultiplikation ausdrücken ?

Matrizenmultiplikation

Kann man Translation mit Matrizenmultiplikation ausdrücken?

$$x' = x + dx$$

$$\begin{pmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \\ a_3 & b_3 & c_3 \end{pmatrix} * \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} a_1 * x + b_1 * y + c_1 * z \\ a_2 * x + b_2 * y + c_2 * z \\ a_3 * x + b_3 * y + c_3 * z \end{pmatrix}$$

Homogene Koordinaten

Die Abbildung f bildet jeden Punkt eines n-dimensionalen Koordinatensystems in den zugehörigen Punkt eines n+1-dimensionalen homogenen Koordinatensystems ab:

$$f : (x, y) \rightarrow (x, y, w) \text{ mit } w \neq 0$$
$$f^-1 : (x, y, w) \rightarrow (x/w, y/w) \text{ mit } w \neq 0$$

Spezialfall $w = 1$:

$$f : (x, y) \rightarrow (x, y, 1)$$
$$f^-1 : (x, y, 1) \rightarrow (x, y)$$

Translationen mit homogenen Koordinaten

Verschiebung eines Punktes: $x' = x + dx, y' = y + dy, z' = z + dz, w' = 1$

$$P' = T * P$$

$$\begin{pmatrix} x' \\ y' \\ z' \\ w' \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & dx \\ 0 & 1 & 0 & dy \\ 0 & 0 & 1 & dz \\ 0 & 0 & 0 & 1 \end{pmatrix} * \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Verschiebung des Koordinatensystems:

$$\begin{pmatrix} x' \\ y' \\ z' \\ w' \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & -dx \\ 0 & 1 & 0 & -dy \\ 0 & 0 & 1 & -dz \\ 0 & 0 & 0 & 1 \end{pmatrix} * \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Transformationen mit homogenen Koordinaten

Skalierung:

$$\begin{pmatrix} x' \\ y' \\ z' \\ w' \end{pmatrix} = \begin{pmatrix} sx & 0 & 0 & \mathbf{0} \\ 0 & sy & 0 & \mathbf{0} \\ 0 & 0 & sz & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{1} \end{pmatrix} * \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Drehung um X-Achse:

$$\begin{pmatrix} x' \\ y' \\ z' \\ w' \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & \mathbf{0} \\ 0 & cos\ \phi & -sin\ \phi & \mathbf{0} \\ 0 & sin\ \phi & cos\ \phi & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{1} \end{pmatrix} * \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Komplexe Transformationen

Rotiere mit R, dann verschiebe mit T_1,
dann skaliere mit S und verschiebe mit T_2

ohne homogene Koordinaten:

$$P' = T_2 + S * (T_1 + R * P)$$

mit homogenen Koordinaten:

$$P' = T_2 * S * T_1 * R * P = M * P$$

Beispiel in 2D

- Rotiere einen Punkt P um 45° um das Zentrum (5,5).

- Vorgehensweise:
  - Verschiebe das Koordinatensystem von (0,0) nach (5,5)
  - Rotiere um 45°
  - Verschiebe das Koordinatensystem zurück zum Ursprung, also von (5,5) nach (0,0)

$$R = \begin{pmatrix} 0.707 & -0.707 & 0 \\ 0.707 & 0.707 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$T_1 = \begin{pmatrix} 1 & 0 & -5 \\ 0 & 1 & -5 \\ 0 & 0 & 1 \end{pmatrix}$$

$$T_2 = \begin{pmatrix} 1 & 0 & 5 \\ 0 & 1 & 5 \\ 0 & 0 & 1 \end{pmatrix}$$

$$P' = T_2 * R * T_1 * P = M * P$$

$$M = \begin{pmatrix} 0.707 & -0.707 & 5 \\ 0.707 & 0.707 & -2.07 \\ 0 & 0 & 1 \end{pmatrix}$$

Beispiel in 2D

- Rotiere einen Punkt P um 45° um das Zentrum (5,5).

- Vorgehensweise:
  - Verschiebe das Koordinatensystem von (0,0) nach (5,5)
  - Rotiere um 45°
  - Verschiebe das Koordinatensystem zurück zum Ursprung, also von (5,5) nach (0,0)

Vorlesung: Grundlagen der Computergrafik

Kapitel 3: 3D-Computergrafik



Kapitel 3.3:

Projektionen

Vorlesung im Sommersemester 2026
Prof. Dr. Stephan Diehl, Informatik, Universität Trier



Historische Beispiele

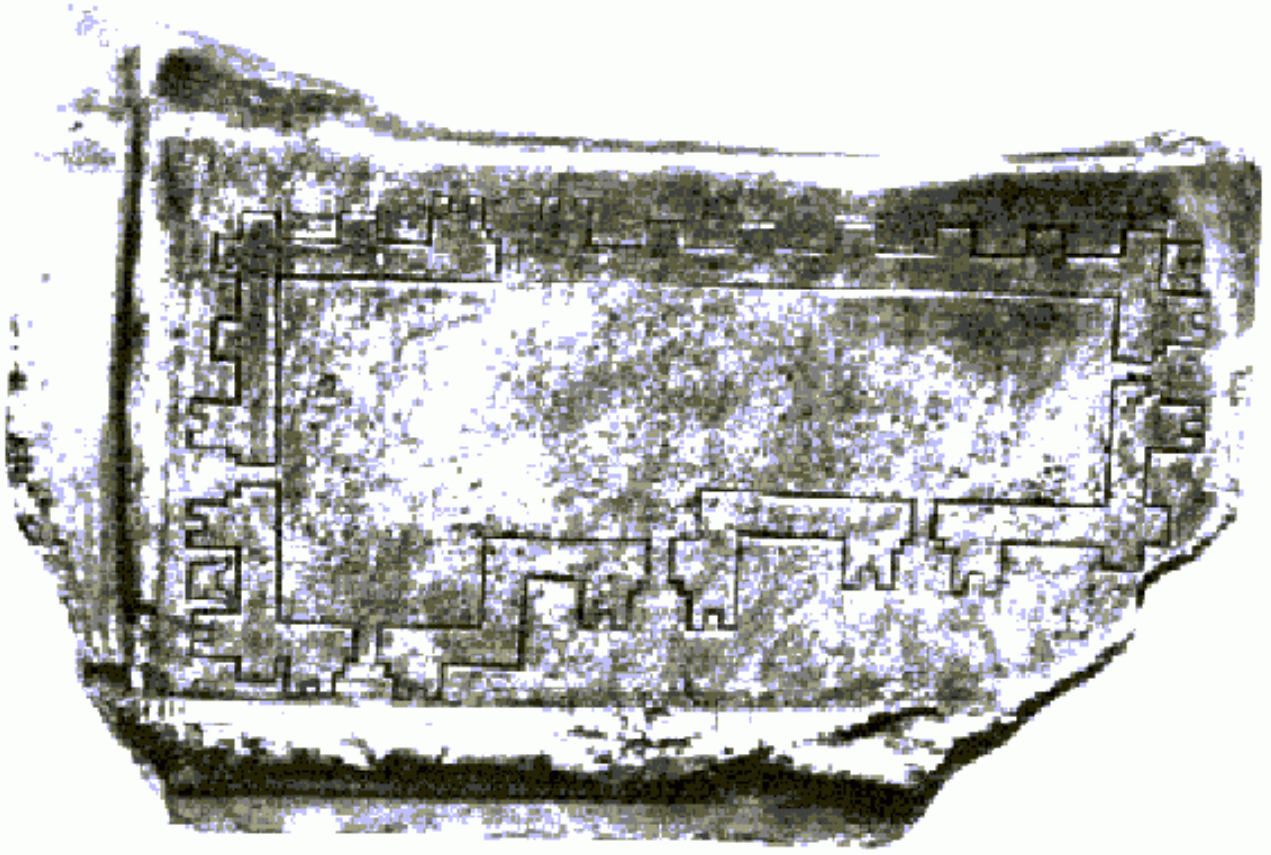


Schattenriss

Ausschnitt aus Karl Friedrich Schinkel,
Die Erfindung des Zeichnens, 1830

Historische Beispiele

- Orthografische Projektion
  - Spezialfall der parallelen Projektion:
    - alle Projektionslinien sind senkrecht zur Projektionsebene und damit parallel.
  - Gebäudeplan aus Mesopotamien (2150 v. Chr.)



Historische Beispiele

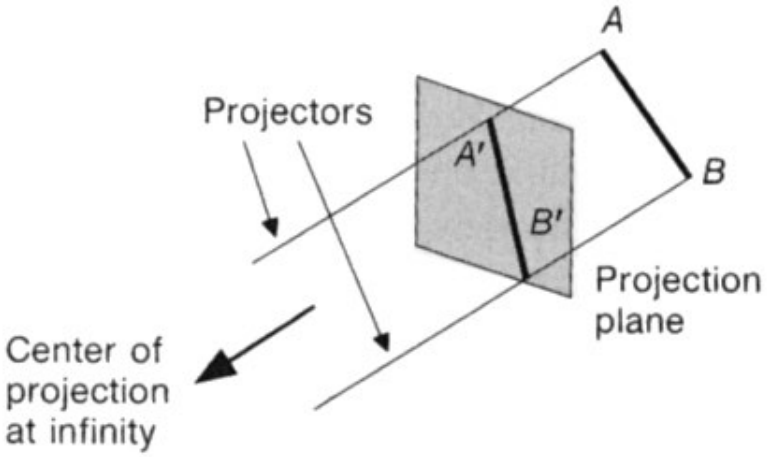
- Perspektivische Projektion
  - Gemälde von Giotto (ca. 1295-1300, Assisi, obere Basilika)

… später mehr dazu



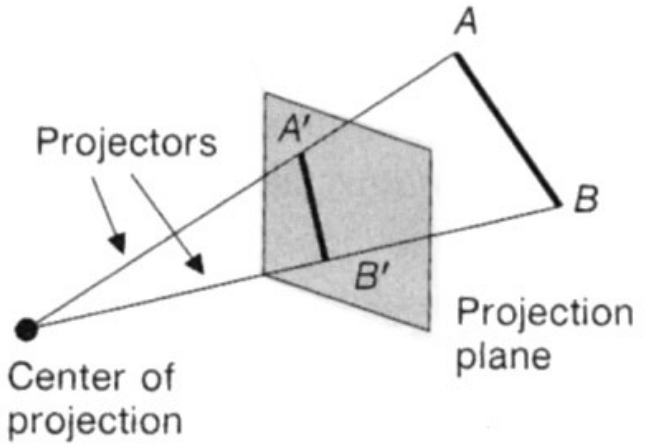
Projektionsarten

parallel



- Projektionszentrum liegt im Unendlichen, daher wird nur die **Projektionsrichtung** angeben.
- Projektionsstrahlen verlaufen parallel zueinander
- keine perspektivische Verkürzung
- Projektion heißt **rechtwinklig** falls Projektionsrichtung der Normalen der Projektionsebene entspricht, sonst **schief**.

perspektivisch

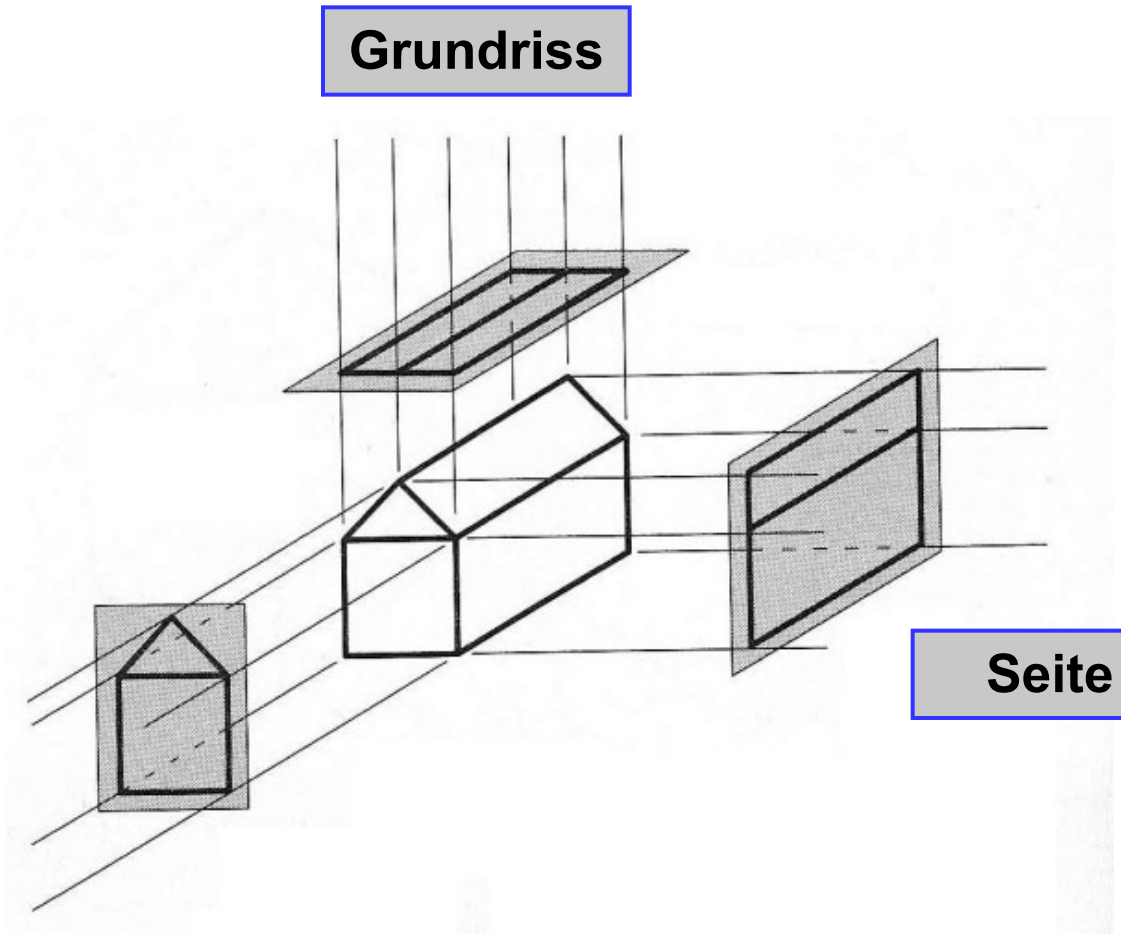


- parallele Geraden, die nicht parallel zur Projektionsebene sind, konvergieren im **Projektionszentrum**
- Größe von Objekten verringert sich mit dem Abstand
- wird auch als **Zentralprojektion** bezeichnet.

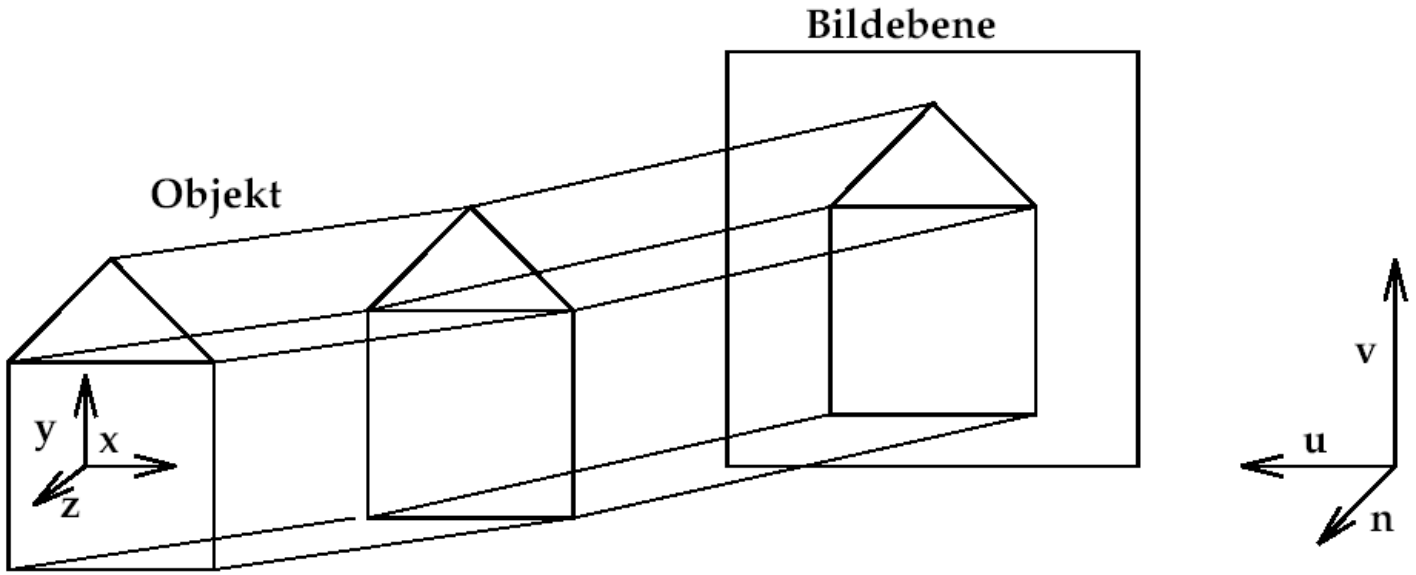
Orthogonalprojektion

Tafelprojektionen:

Projektionsebenen parallel zu Ebenen des Koordinatensystems



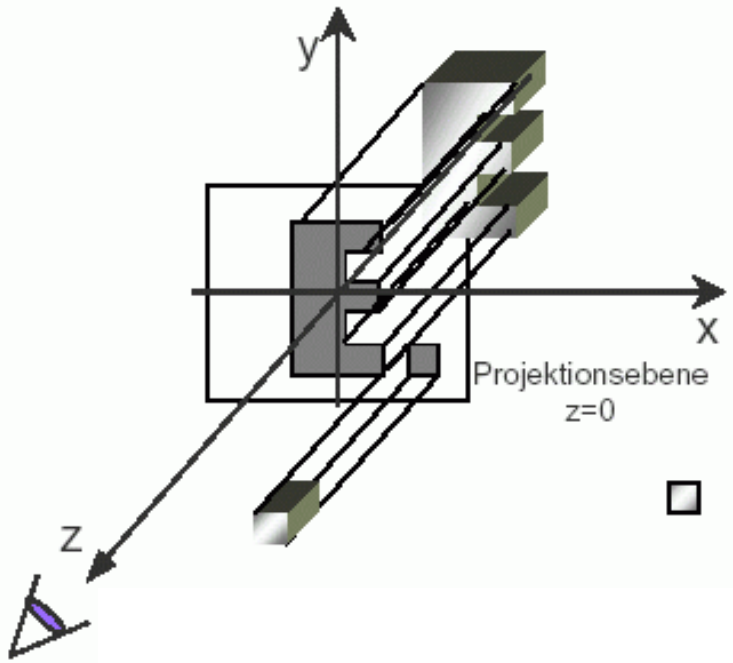
Orthogonalprojektion



Einfachster Fall: Orthogonalprojektion

$$(x, y, z) \rightarrow (x, y)$$

Orthogonalprojektion



$$\mathbf{P}_0 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Die Z-Komponente wird auf Null gesetzt, d.h. ignoriert.
- **Achtung:**
  - Positive und negative Z-Werte werden auf die XY-Ebene abgebildet.
  - Auch Objekte, die hinter dem Betrachter sind, werden auf die Bildebene abgebildet.

Perspektivische Projektion

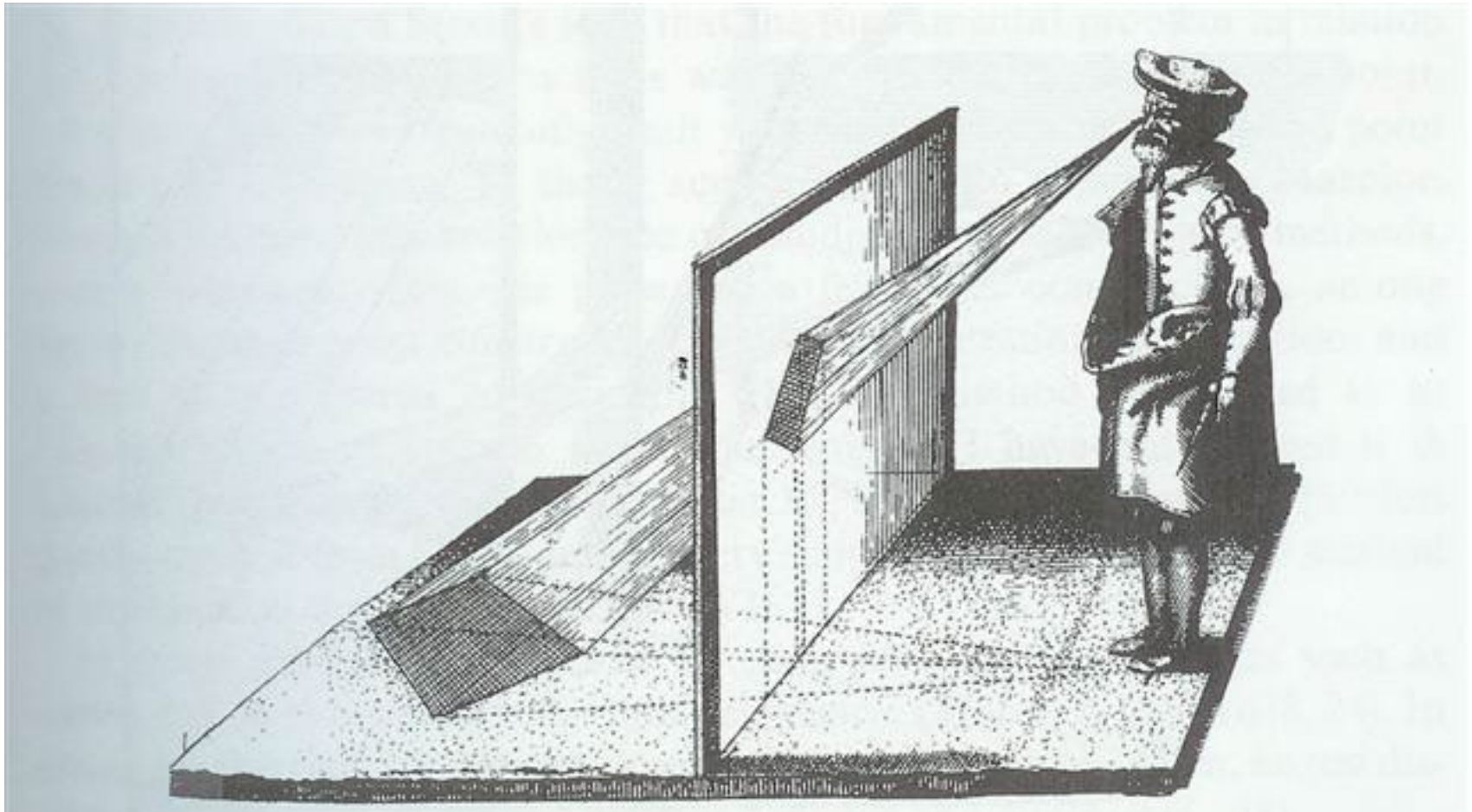
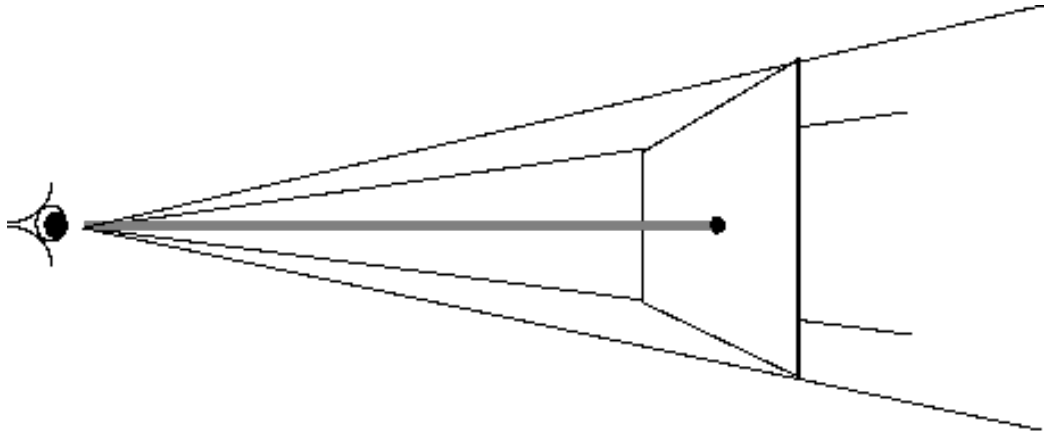


FIGURE VII.7. Marolois's illustration of the perspective model. Engraving by Hendrik Hondius, Marolois 1614, figure 7.

Perspektivische Projektion

Leon Battista Alberti (1404-1472) veröffentlichte erste Abhandlung über Perspektive (Della Pittura) im Jahre 1435**:**

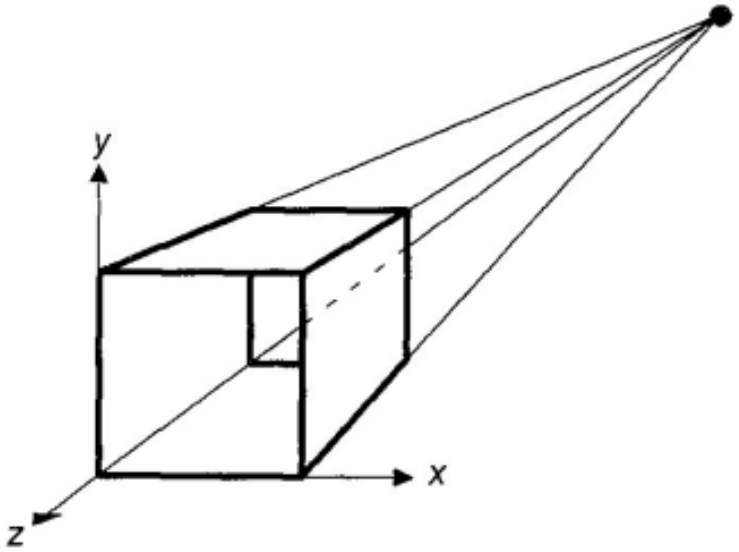
– "a **painting** is the **intersection** of a **visual pyramid** at a given distance, with a fixed center and a defined position of light, represented by art with lines and colors on a given **surface**." *(Alberti, On Painting pp.32-3)*

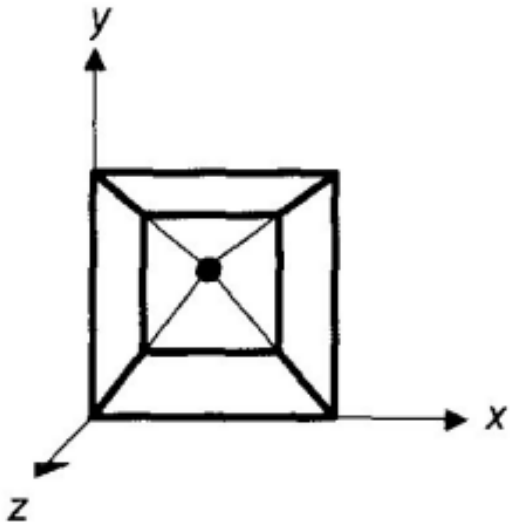


Leono Battista Alberti (1404-1472)

Perspektivische Projektion

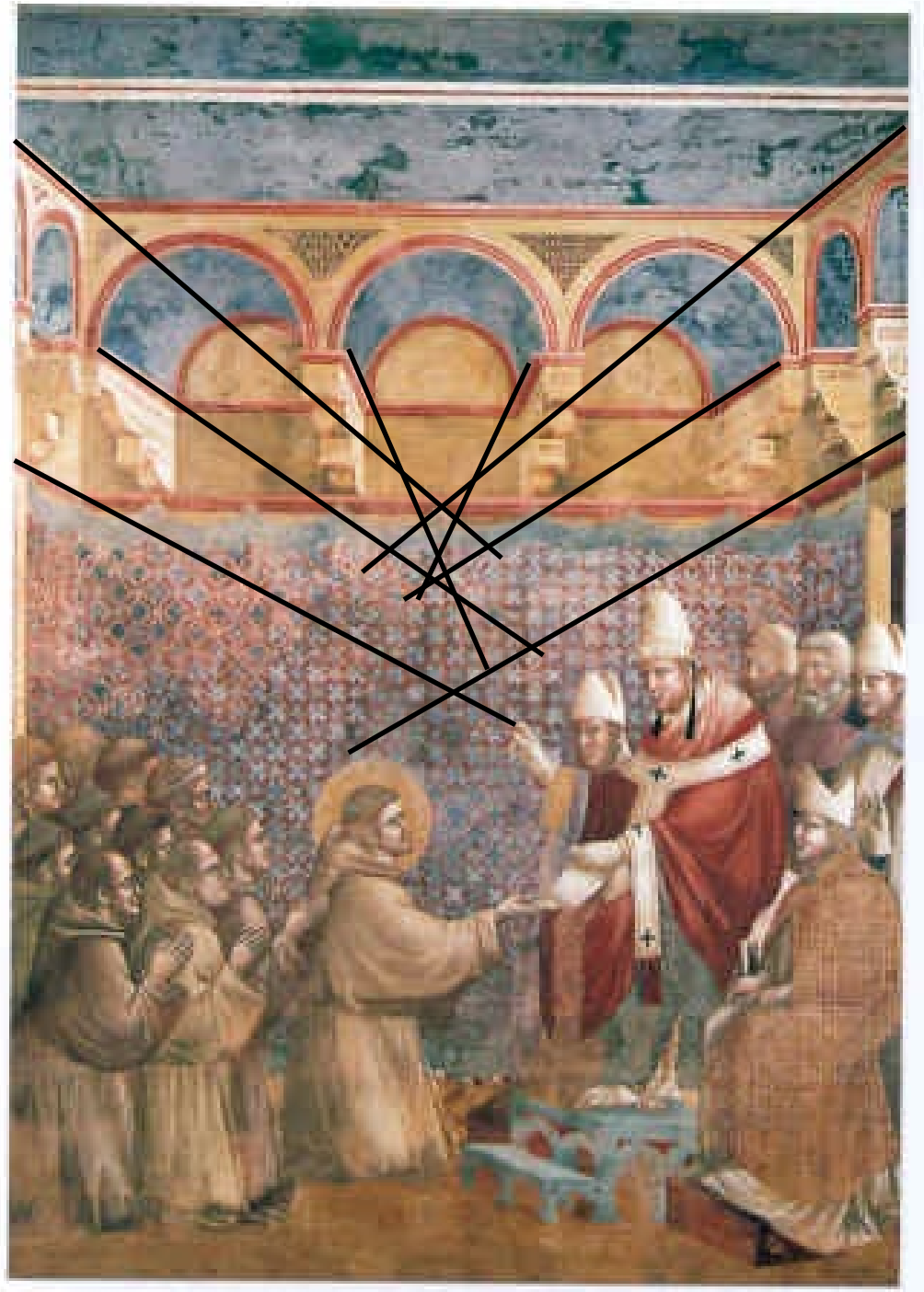
Parallele Linien des Objekts konvergieren zum **Projektionszentrum (Fluchtpunkt),** wenn sie nicht parallel zur Projektionsebene sind.







Fluchtpunkte?

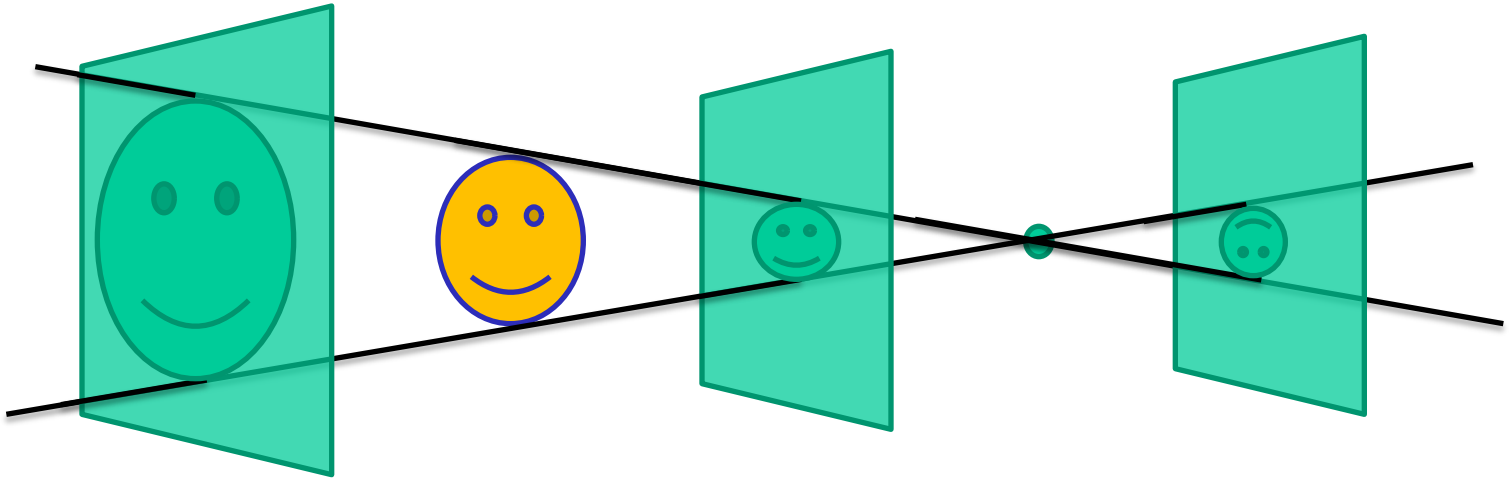


Noch nicht systematisch:

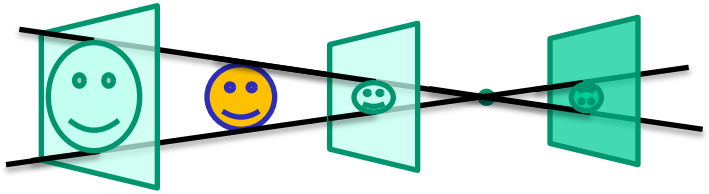
- Linien konvergieren nicht in einzelnem Fluchtpunkt

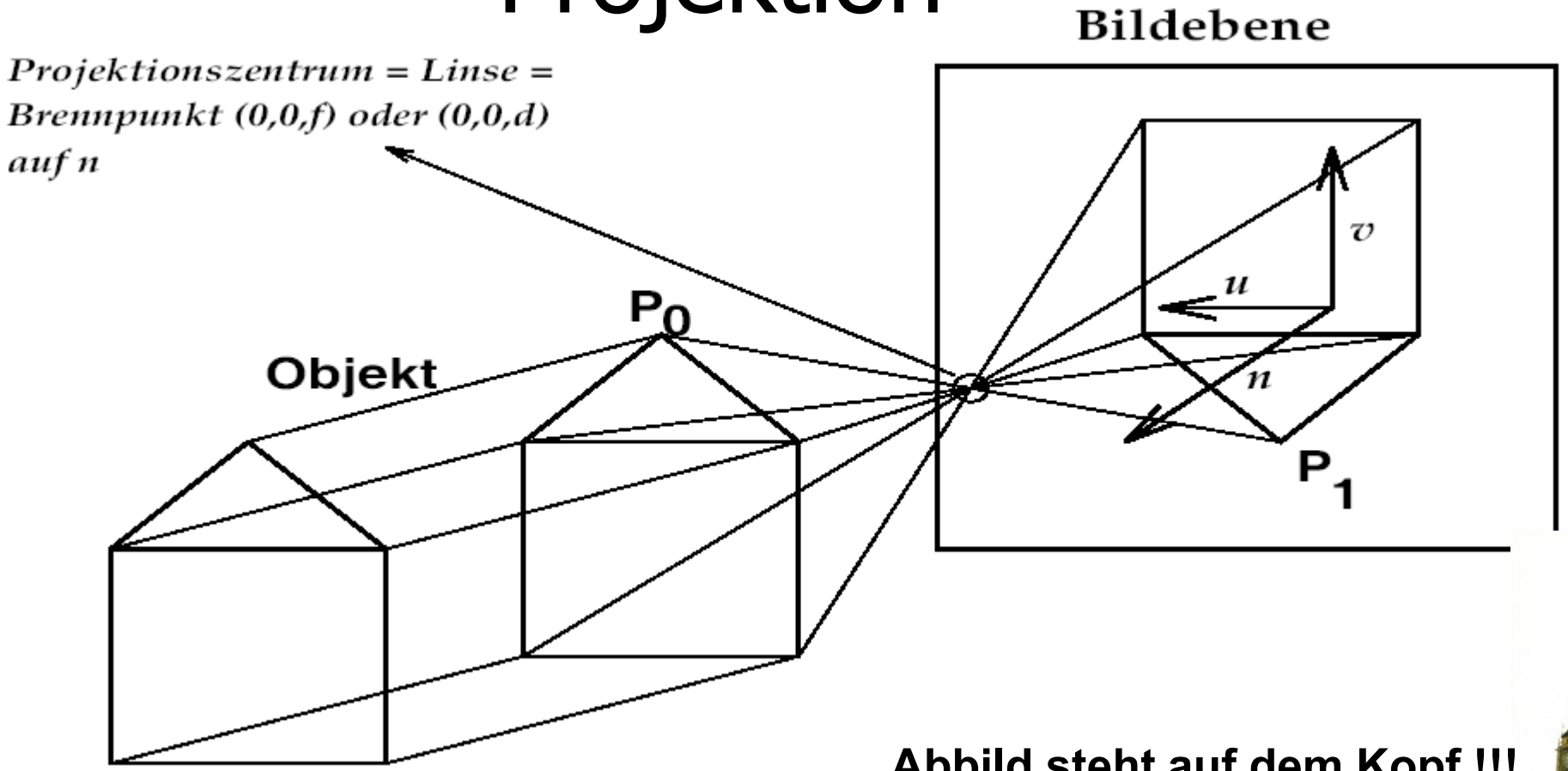
Perspektivische Projektion

Lage der Projektionsebene



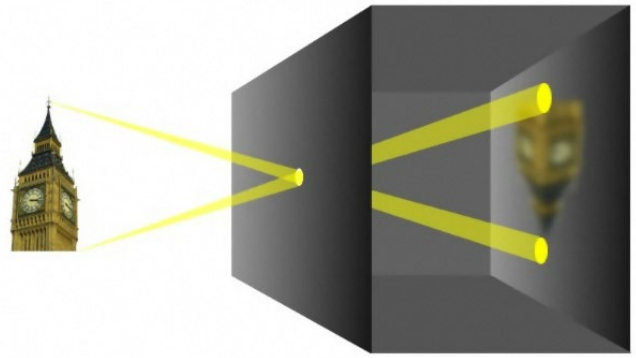
Perspektivische Projektion



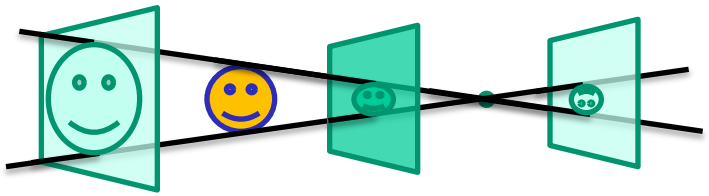


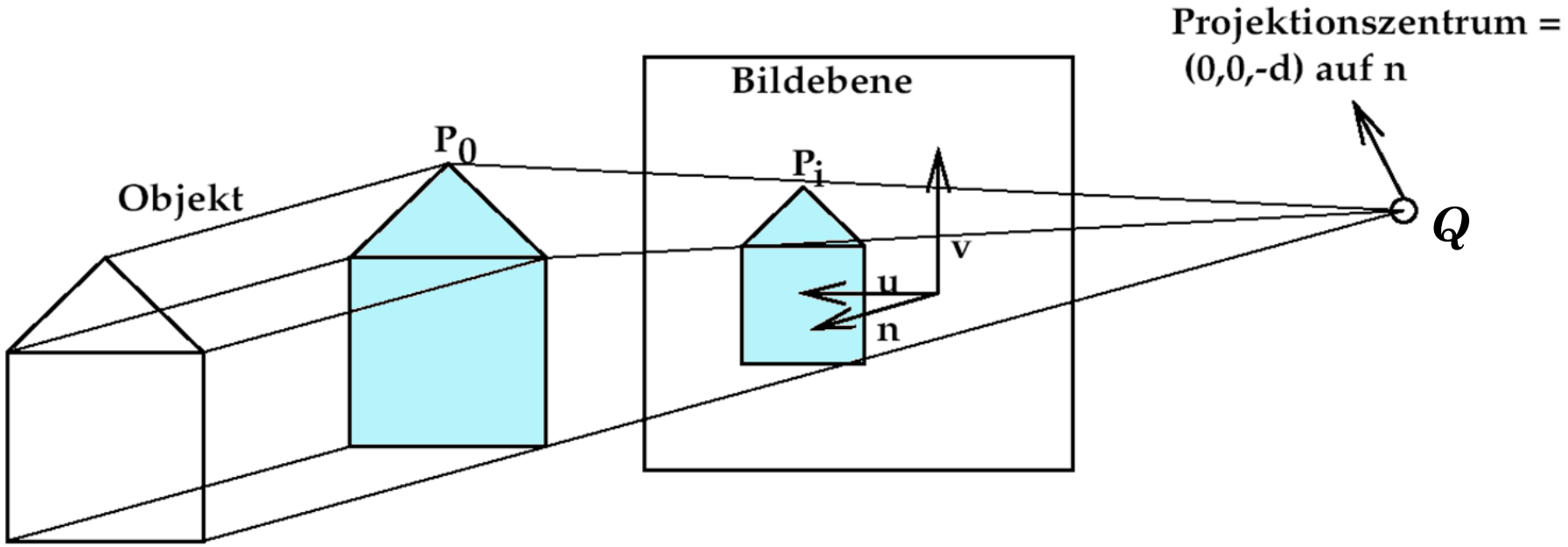
Lochkamera oder Auge

Abbild steht auf dem Kopf !!!



Perspektivische Projektion





$$P_0 = Q + \alpha * (P_i - Q)$$

Projektionen aus beliebigen Richtungen und mit beliebigem Projektionszentrum (Blickpunkt) können durch Translationen und Drehungen in diese Form überführt werden !!!

Perspektivische Projektion

$$P_0 = Q + \alpha * (P_i - Q)$$

$$P_0 = \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ -d \end{pmatrix} + \alpha * \left(\begin{pmatrix} u \\ v \\ 0 \end{pmatrix} - \begin{pmatrix} 0 \\ 0 \\ -d \end{pmatrix} \right)$$

Perspektivische Projektion

$$P_0 = Q + \alpha * (P_i - Q)$$

$$P_0 = \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ -d \end{pmatrix} + \alpha * \left(\begin{pmatrix} u \\ v \\ 0 \end{pmatrix} - \begin{pmatrix} 0 \\ 0 \\ -d \end{pmatrix} \right)$$

Für die Komponenten:

$$x = \alpha * u$$

$$y = \alpha * v$$

$$z = -d + \alpha * d \;\Rightarrow\; \alpha = \frac{z+d}{d}$$

$$x = \frac{z+d}{d} * u \;\Rightarrow\; u = \frac{x*d}{z+d}$$

$$y = \frac{z+d}{d} * v \;\Rightarrow\; v = \frac{y*d}{z+d}$$

$$u = \frac{x*d}{z+d}$$
$$v = \frac{y*d}{z+d}$$

Perspektivische Projektion

Matrixdarstellung mit homogene Koordinaten:

$$\begin{pmatrix} u \\ v \\ n \\ w \end{pmatrix} = \begin{pmatrix} d & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & d \end{pmatrix} * \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

oder

$$\begin{pmatrix} u \\ v \\ n \\ w \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{d} & 1 \end{pmatrix} * \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Perspektivische Projektion

$$u = \frac{x*d}{z+d}$$
$$v = \frac{y*d}{z+d}$$

Orthogonale Projektion ergibt sich mit Hilfe der Regel von l'Hospital für $d \to \infty$:

$$u = \lim_{d\to\infty} \frac{x*d}{z+d} = \lim_{d\to\infty} \frac{(x*d)'}{(z+d)'} = x$$

$$v = \lim_{d\to\infty} \frac{y*d}{z+d} = \lim_{d\to\infty} \frac{(y*d)'}{(z+d)'} = y$$

Regel von l'Hopital: Grenzwerte von Funktionen, die sich als Quotient zweier gegen 0 konvergierender Funktionen schreiben lassen, lassen sich mithilfe der ersten Ableitungen dieser Funktionen berechnen:
Falls die Grenzwerte $\lim_{x\to x_0} f(x)$ und $\lim_{x\to x_0} g(x)$ beide 0 oder beide ∞ sind,

dann gilt $$\lim_{x\to x_0} \frac{f(x)}{g(x)} = \lim_{x\to x_0} \frac{f'(x)}{g'(x)}$$

Parallelprojektion

Auch die Parallelprojektion lässt sich als Matrix darstellen.

Matrixdarstellung einer schiefwinkligen Parallelprojektion mit der *xy*-Ebene als Projektionsebene und Projektionsstrahlen mit Richtungsvektor (*rx*, *ry*, *rz*):

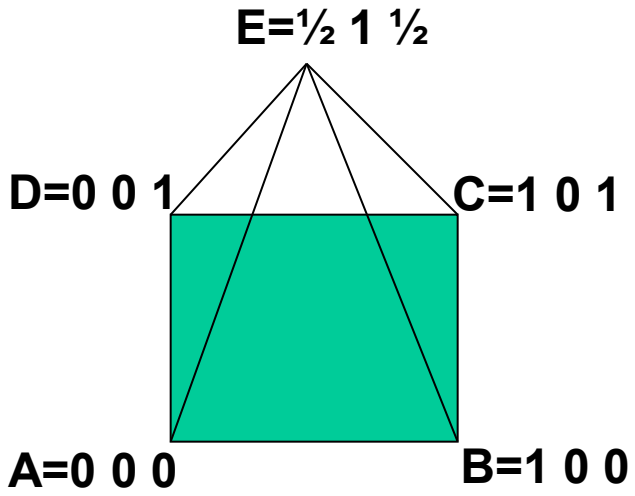
$$P_{obl} = \begin{pmatrix} 1 & 0 & -rx/rz & 0 \\ 0 & 1 & -ry/rz & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Ohne Herleitung !

Beispiel

Gegeben seien die 5 Eckpunkte einer Pyramide:

$$\begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}, \begin{pmatrix} \frac{1}{2} \\ 1 \\ \frac{1}{2} \end{pmatrix}$$



Drehe die Pyramide um 45° um die Y-Achse
und projiziere perspektivisch mit $d = 10$:

$$P' = \begin{pmatrix} d & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & d \end{pmatrix} * \begin{pmatrix} cos\ \phi & 0 & sin\ \phi & 0 \\ 0 & 1 & 0 & 0 \\ -sin\ \phi & 0 & cos\ \phi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} * P$$

Für Drehung der Pyramide um die eigene Achse muss zusätzlich der Ursprung verschoben werden!